GUÍA MAESTRA DEL EXPOSITOR

Manual Pedagógico para la Comprensión y Sustentación Oral del Proyecto

Tema: Planificación de CPU • Gestión de Memoria Virtual con Paginación Pura
Estudiantes: Carlos Andrés Ibañez Barrios • Shaadon Jiménez Flores
Docente: Mag. Sergio Luis Lubo Argumedo
Universidad Cooperativa de Colombia • Facultad de Ingenierías • Santa Marta • 2026

¿Para qué sirve este documento?
A diferencia del *Informe Técnico* (formal, denso y lleno de código), este manual fue redactado en lenguaje claro y didáctico para que **entiendas exactamente qué hace cada parte del sistema**, qué significan todos los números que viste en la terminal, cómo responder las preguntas difíciles del profesor y cómo explicar con autoridad técnica cada decisión de diseño tomada en el simulador.

1. Glosario Maestro de Abreviaciones, Siglas y Fórmulas

En los exámenes y sustentaciones de Sistemas Operativos, dominar la terminología exacta transmite seguridad y rigor. A continuación se presenta la totalidad de siglas empleadas en las tablas de resultados:

Sigla	Nombre Formal	Significado Intuitivo (¿Qué representa?)	Fórmula o Criterio
TAT	Turnaround Time (Tiempo de Retorno)	El tiempo total desde que el proceso llega al sistema hasta que finaliza por completo su última ráfaga.	**TAT = T_fin - T_llegada**
WT	Waiting Time (Tiempo de Espera)	El tiempo que el proceso pasa 'haciendo fila' en la cola de LISTOS esperando que le den CPU.	**WT = Ticks en estado LISTO** = TAT - T_CPU - T_IO
RT	Response Time (Tiempo de Respuesta)	Tiempo que tarda el proceso en recibir la CPU por **primera vez**. Mide la agilidad interactiva del sistema.	**RT = T_primer_despacho - T_llegada**
CS	Context Switch (Cambio de Contexto)	Acción del SO de guardar el estado (PCB) del proceso actual en memoria y restaurar el del siguiente.	**costo_cs = 1 tick** (real) costo_cs = 0 tick (ideal)
σ_WT	Desviación Estándar de WT (Equidad)	Mide qué tan dispersas están las esperas entre procesos. Un valor bajo significa que el trato fue parejo.	**sqrt(Σ(WT_i - μ)^2 / N)**
Inanición	Starvation (Inanición Estadística)	Alerta cuando un proceso espera desproporcionadamente más de lo que dura su propio cómputo.	**Alerta si: WT / T_CPU ≥ 4.0**
PCB	Process Control Block (Bloque de Control)	Estructura de datos que almacena el 'carnet de identidad' del proceso: PID, estado, ráfagas y tabla de páginas.	Clase **Proceso** en pcb.py
MMU	Memory Management Unit (Unidad de Manejo)	Dispositivo de hardware simulado que convierte direcciones virtuales en direcciones físicas en RAM.	**dir_fisica = (marco * 4096) + offset**

HR	Hit Rate (Tasa de Aciertos)	Porcentaje de veces que la página solicitada ya estaba cargada en memoria RAM sin necesidad de disco.	**HR = (Aciertos / Total) * 100%**
PFR	Page Fault Rate (Tasa de Fallos)	Porcentaje de accesos donde la página NO estaba en RAM y el SO tuvo que traerla (Fallo de Página).	**PFR = 100% - HR**
IF	Fragmentación Interna	Espacio desperdiciado dentro del último marco asignado porque el proceso pidió menos de 4096 bytes.	**mem_asignada - mem_necesitada**
EF	Fragmentación Externa	Memoria libre total suficiente pero dispersa en huecos. En paginación pura **es exactamente 0**.	**0 bytes** (por diseño de marcos fijos)

2. ¿Cómo Funciona la Planificación de CPU? (Explicación Intuitiva)

El procesador solo puede ejecutar las instrucciones de **un solo proceso a la vez**. Cuando varios programas quieren correr simultáneamente, el sistema operativo necesita una política para decidir a quién le toca el turno. Nuestro simulador implementa las 3 estrategias clásicas:

2.1. Los 3 Algoritmos Explicados de Forma Simple

1. FCFS (First-Come, First-Served):
• *Metáfora:* Es la fila tradicional del banco. Quien llega primero, se atiende primero.
• *Ventaja:* Muy fácil de programar y casi no gasta tiempo cambiando de contexto.
• *El gran peligro (Efecto Convoy):* Si el primer cliente de la fila va a pagar 50 facturas (un proceso gigante de CPU=20), los clientes de atrás que solo iban a retirar un recibo de 1 minuto tienen que esperar 20 minutos parados. Eso arruina la interactividad.

2. SJF No Expropiativo (Shortest Job First):
• *Metáfora:* En la fila rápida del supermercado, quien tiene menos artículos pasa primero.
• *Ventaja:* Minimiza matemáticamente el tiempo de espera promedio general.
• *Desempate:* Si dos procesos tienen la misma ráfaga, pasa el que llegó primero; si llegaron al mismo tiempo, el de menor PID.
• *Peligro:* Si siguen llegando procesos cortitos todo el tiempo, un proceso largo nunca recibe CPU (Inanición).

3. Round Robin con Quantum q:
• *Metáfora:* Un reloj de ajedrez donde cada jugador tiene exactamente *q* segundos de turno. Si no termina su jugada en ese tiempo, se le pausa y pasa al final de la fila.
• *Ventaja:* Nadie monopoliza la CPU. Todos los procesos reciben atención rápidamente (bajísimo Tiempo de Respuesta RT).
• *Regla de desempate clave:* Si un proceso agota su quantum en el mismo instante en que llega un nuevo proceso, el simulador **le da prioridad al nuevo proceso**, colocándolo de primero en la cola para que su tiempo de respuesta inicial sea excelente.

■■ ***Clave para la Sustentación: El cambio de contexto no es gratis:*** *Salvar los registros y el estado del PCB de un proceso que sale y cargar el del nuevo consume cómputo real. En el simulador esto se modela como costo_cs = 1 tick. Si el quantum es muy pequeño (ej. q=2), el procesador pasa demasiado tiempo conmutando en lugar de trabajar.*

3. ¿Cómo Funciona la Gestión de Memoria? (Paginación y MMU)

En los sistemas primitivos, a un proceso se le asignaba un bloque continuo de memoria RAM. Si los procesos se abrían y cerraban, la RAM quedaba llena de huecos dispersos que no alcanzaban para nadie (**Fragmentación Externa**). La solución moderna es la **Paginación Pura**:

3.1. Páginas, Marcos y el Algoritmo MMU en 4 Pasos

• **Páginas:** Bloques lógicos de tamaño fijo (4 KB = 4096 bytes) en el espacio del programa.
• **Marcos (Frames):** Bloques del mismo tamaño fijo (4 KB) pero en la memoria física real (RAM).
• **La MMU:** Cada vez que la CPU quiere leer una dirección virtual, ejecuta este algoritmo canónico de 4 pasos:

Paso	Operación MMU	Ejemplo Numérico Real (dir_virtual = 9500 B)
1	**Número de Página (p):** p = dir_virtual // 4096	9500 // 4096 = **Página 2**
2	**Desplazamiento / Offset (d):** d = dir_virtual % 4096	9500 % 4096 = **1308 bytes** de desplazamiento
3	**Validación de Límites:** Verificar si p < len(tabla_paginas)	Si el proceso tiene 3 páginas, 2 < 3 → **Válida (Marco #5)**. Si pide página 6 → **Detona Segmentation Fault**.
4	**Dirección Física Final:** dir_fisica = (id_marco * 4096) + d	(5 * 4096) + 1308 = **21,788 bytes** en la RAM real.

3.2. Fragmentación y Políticas de Reemplazo (FIFO vs LRU)

• **Fragmentación Externa = 0 bytes:** Como cualquier página de 4 KB cabe en cualquier marco de 4 KB disponible, no importa si están separados. No hay huecos desperdiciados.
• **Fragmentación Interna:** Si un proceso pide 9,000 bytes, necesita 3 páginas de 4096 (12,288 bytes). La diferencia 12,288 - 9,000 = 3,288 bytes se desperdicia dentro del último marco asignado.
• **Reemplazo ante Fallo de Página (Page Fault):** Cuando la RAM se llena y se pide una página nueva, hay que expulsar a una víctima:
→ **FIFO:** Desaloja la página que llegó primero a la RAM (la más vieja). Es ciega, pues puede botar una página que el proceso está usando intensamente.
→ **LRU:** Desaloja la página que lleva más tiempo sin ser usada (la menos recientemente referenciada). Se basa en el **Principio de Localidad Temporal**.

4. Guía de Lectura: ¿Qué Significan los Resultados que Obtuviste?

Aquí está el análisis directo de los números exactos que salieron en tu terminal al ejecutar main.py. Léelo con atención para saber qué decirle al profesor cuando mire cada tabla:

4.1. Escenario 1: Carga Ligera (Con E/S y Bloqueos)

• **FCFS:** TAT=13.75, WT=9.00, RT=4.50, CS=5, Inanición=0.
• **SJF:** TAT=12.75, WT=8.00, RT=6.75, CS=5, Inanición=1.
• **Round Robin (Q=3):** TAT=16.25, WT=11.50, RT=4.00, CS=6, Inanición=0.

¿Cómo explicarlo?

1. *SJF es el rey de la eficiencia promedio:* Logra el menor tiempo de retorno (12.75) y menor espera (8.00) porque despacha de primero a los procesos cortos.
2. *Round Robin es el rey de la interactividad:* Aunque su TAT es mayor por conmutar turnos, su tiempo de respuesta es el más rápido de todos (RT=4.00).
3. *¿Por qué aquí WT es mayor que RT?* Porque P1 y P3 realizan transferencias de E/S. Pasan a BLOQUEADO y, cuando regresan a LISTO, tienen que volver a esperar en la fila, acumulando más tiempo de espera (WT) que el que tardaron en su primer despacho (RT).

4.2. Escenario 2: El Efecto Convoy (La Prueba de Fuego)

• **FCFS:** TAT=24.20, WT=18.60, RT=18.60, CS=4, Inanición=4.
• **SJF:** TAT=23.40, WT=17.80, RT=17.80, CS=4, Inanición=4.
• **Round Robin (Q=2):** TAT=15.40, WT=9.80, RT=5.00, CS=7, Inanición=3.

¿Cómo explicarlo?

1. *El colapso de FCFS:* El proceso P1 (ráfaga de 20 ticks) acapara el procesador en t=0. Los procesos cortos P2, P3, P4 y P5 quedan atrapados esperando casi 20 ticks solo para empezar. Su tiempo de respuesta es altísimo (18.60) y 4 de los 5 procesos caen en inanición.
2. *El rescate de Round Robin:* Al cortarle la CPU a P1 cada 2 ticks (Q=2), los procesos cortos terminan rapidísimo. El tiempo de respuesta cae de 18.60 a **5.00 ticks** (¡una reducción brutal del **73.1%**!) y el TAT promedio baja de 24.20 a 15.40.
3. *En Memoria (LRU vence a FIFO):* La memoria se limitó a 4 marcos para forzar reemplazos sobre el patrón canónico de Silberschatz. **LRU logró 46.43% de aciertos frente al 39.29% de FIFO** (13 aciertos contra 11, y solo 4 reemplazos contra 6). Esto demuestra que recordar las páginas usadas recientemente ahorra fallos de página.

4.3. Escenario 3: La Sensibilidad al Quantum y el Costo del Context Switch

• **Con costo real (CS=1 tick):**
  → RR Q=2: TAT=45.38, WT=39.12, RT=9.62, **CS=25**.
  → RR Q=4: TAT=34.62, WT=28.38, RT=14.75, **CS=14**.
  → RR Q=8: TAT=31.00, WT=24.75, RT=19.50, **CS=9**.
• **En mundo ideal sin costo de hardware (CS=0 tick):**
  → RR Q=2: TAT=28.62, WT=22.38, RT=5.75, **CS=25**.

¿Cómo explicarlo?

Este escenario enseña el **Trade-off fundamental del Quantum**:
Si usas un quantum muy pequeño (Q=2), la CPU cambia 25 veces de proceso. En un sistema real (CS=1), esos 25 cambios le roban 25 ticks a la CPU, inflando el TAT de 28.62 a 45.38. Si usas Q=8, los cambios bajan a 9 y el TAT mejora a 31.00, pero el tiempo de respuesta empeora a 19.50 porque se parece a FCFS.

■ ***Clave para la Sustentación: La prueba matemática de los tests verdes [TEST WT=RT]:***
En el Escenario 2, FCFS dio WT=18.60 y RT=18.60 (iguales). SJF dio WT=17.80 y RT=17.80 (iguales).
¿Por qué? Porque como son algoritmos no expropiativos y los procesos no hacen E/S, una vez que un proceso entra a la CPU corre hasta morir. Todo lo que esperó antes de entrar (RT) es exactamente todo lo que esperó en toda su vida (WT). Que ambos números den exactamente idénticos prueba que tu simulador cuenta la sobrecarga de conmutación con cero error.

5. Banco de Preguntas Típicas del Jurado y Respuestas de 10/10

Estas son las preguntas más comunes que un profesor de Sistemas Operativos suele hacer en este proyecto. Aprende el concepto detrás de cada respuesta:

Profesor: ¿Por qué en el Escenario 2 el tiempo de espera promedio (WT) es matemáticamente igual al tiempo de respuesta (RT) en FCFS y SJF?

Respuesta: Porque en el Escenario 2 los algoritmos son NO expropiativos y los procesos tienen cómputo puro sin operaciones de E/S. Esto significa que un proceso solo espera una vez en la cola de listos; en el momento en que es despachado a la CPU, ejecuta hasta terminar sin volver a esperar jamás. Por ende, el tiempo transcurrido hasta su primer despacho (RT) es idéntico a su tiempo de espera total acumulado (WT).

Profesor: ¿Y por qué en el Escenario 1 esa igualdad WT == RT ya no se cumple?

Respuesta: Porque en el Escenario 1 los procesos P1 y P3 realizan operaciones de E/S intermedias. Cuando piden E/S pasan a estado BLOQUEADO y, al terminar la transferencia periférica, regresan a la cola de LISTOS. Esa segunda espera se suma al WT total, haciendo que WT sea mayor que el RT inicial.

Profesor: ¿Qué pasa si el quantum (q) de Round Robin se hace extremadamente grande?

Respuesta: Si el quantum es mayor o igual que la ráfaga más larga de todos los procesos, ningún proceso será expropiado por límite de tiempo. En ese límite matemático, Round Robin degenera exactamente en el algoritmo FCFS (First-Come, First-Served).

Profesor: ¿Por qué LRU le ganó a FIFO en el Escenario 2, pero en el Escenario 3 empataron?

Respuesta: En el Escenario 2, el proceso P1 ejecutó el patrón de referencias de Silberschatz en una memoria restringida a 4 marcos. Ese patrón tiene localidad temporal (reutiliza páginas consultadas recientemente). LRU protege esas páginas calientes, mientras que FIFO las desaloja por ser viejas. En cambio, en el Escenario 3 hay 8 procesos rotando rápidamente en Round Robin; esa rotación constante satura la RAM y hace que el orden cronológico de llegada a la memoria (FIFO) coincida con el orden de recencia (LRU).

Profesor: ¿Cómo garantiza su simulador que no exista fragmentación externa en memoria?

Respuesta: Por la propia definición de la Paginación Pura. La memoria física se divide en marcos de tamaño fijo (4096 bytes) y el espacio virtual en páginas del mismo tamaño. Cualquier página libre puede colocarse en cualquier marco disponible sin importar que no sean contiguos. Al no requerir memoria contigua, la fragmentación externa es estrictamente 0 bytes.

Profesor: ¿Qué hace su simulador si un proceso intenta acceder a una dirección virtual fuera de su rango asignado?

Respuesta: El simulador no recorta en silencio la dirección. En el Paso 3 del algoritmo de la MMU, valida si el número de página calculado excede el tamaño de la tabla de páginas del proceso. Si lo excede, detona inmediatamente una excepción formal IndexError etiquetada como 'Violación de Segmento (Segmentation Fault)', emulando con precisión las trampas de hardware reales de un sistema operativo.

6. 'Chuleta' Resumen: Guion de 2 Minutos para Abrir tu Exposición

Si te piden resumir tu proyecto en 2 minutos frente al salón o al profesor, di exactamente esto:

■■ ***Clave para la Sustentación:*** *'Buenos días profesor. Nuestro proyecto consistió en construir un simulador ciclo a ciclo en Python 3 que unifica dos subsistemas fundamentales: la Planificación de CPU y la Gestión de Memoria Virtual por Paginación Pura.*

En la parte de CPU demostramos empíricamente el ***Efecto Convoy*** *en el Escenario 2: con FCFS, un proceso largo acaparó el procesador obligando a los procesos cortos a esperar 18.6 ticks. Al aplicar* ***Round Robin (Q=2)****, el tiempo de respuesta cayó un* ***73.1%*** *(a 5 ticks), resolviendo la interactividad.*

También modelamos el costo real de cambio de contexto (costo_cs = 1 tick), demostrando el trade-off del quantum: quantums muy pequeños aumentan los cambios de contexto (25 CS en el Escenario 3) y disparan el TAT a 45.38 ticks.

En la parte de Memoria implementamos la traducción MMU en 4 pasos, comprobamos ausencia de fragmentación externa (0 bytes), calculamos la fragmentación interna y demostramos cómo el algoritmo ***LRU superó a FIFO logrando un 46.43% de aciertos frente a 39.29%*** *gracias al principio de localidad temporal.'*